



برنامه‌نویسی به زبان C++ (فصل دوم)

مدرس: مریم خراشادیزاده

فهرست مطالب

- فصل اول : مقدمات زبان C++
- فصل دوم : ساختارهای تصمیم گیری و تکرار
- فصل سوم: آرایه ها و رشته ها
- فصل چهارم: توابع
- فصل پنجم: ساختارها و اشاره گرها



دستورالعمل شرطی

Single-selection statement



دستورالعمل شرطی **if**، عبارت درون پرانتز را بررسی نموده و اگر عبارت درست (**true**) ارزیابی شود، یک یا چند دستورالعمل بعد از **if** را اجرا می کند.

```
if (condition)  
    statement;
```

(الف)

```
if (condition )  
{  
    statement 1 ;  
    ...  
    statement n ;  
}
```

(ب)

A blue cloud shape with a black outline. Inside the cloud, the word "if" is written in a stylized, bold, red font.

مثال



مثال ۱: برنامه‌ای بنویسید که عددی را از ورودی دریافت کند و قدرمطلق آن را چاپ کند.

```
#include <iostream>
using namespace std;

int main()
{
    double number; // declare a variable to store the input number
    cout << "Enter a number: "; // display message
    cin >> number; // read a real number from user into number

    // check if the number is negative
    if (number < 0)
        number = -number; // convert the number to its absolute value

    cout << "\n The absolute value is: " << number ; // display the absolute value

    return 0; // indicate that program ended successfully
}
```

دستورالعمل شرطی

Double-selection statement



دستورالعمل شرطی `if ... else` عبارت درون پرانتز را بررسی نموده و اگر عبارت درست (`true`) ارزیابی شود، یک یا چند دستورالعمل بعد از `if` و اگر نادرست (`false`) ارزیابی شود یک یا چند دستورالعمل بعد از `else` را اجرا می کند.

```
if (condition)
    statement;
else
    statement;
```

(الف)

```
if (condition)
{
    statement 1;
    ...
    statement n;
}
else
{
    statement 1;
    ...
    statement m;
}
```

(ب)

if-else

مثال



مثال ۲: برنامه‌ای بنویسید که عددی را از ورودی دریافت کند و تعیین کند که عدد زوج است یا فرد.

```
#include <iostream>
using namespace std;

int main()
{
    int number; // declare a variable to store the input number
    cout << "Enter a number: "; // display message
    cin >> number; // read a real number from user into number

    // check if the number is even or odd
    if (number % 2 == 0) // if the remainder of division by 2 is zero, the number is even
        cout << "\n" << number << " is even.";
    else // if the remainder is not zero, the number is odd
        cout << "\n" << number << " is odd.";

    return 0; // indicate that program ended successfully
}
```

عملگر شرطی

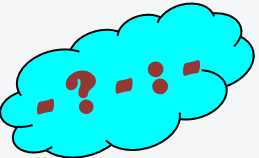
Conditional Operator



شکل کلی عملگر شرطی بصورت زیر می باشد:

condition ? expression _ if_true : expression _ if_false

عملگر شرطی تنها عملگری در C++ می باشد که دارای سه عملوند (ternary operator) می باشد.



مثال



مثال ۳: برنامه‌های محاسبه قدرمطلق و تعیین زوج و فرد را با استفاده از عملگر شرطی بنویسید.

```
int main()
{
    double number;
    cout << "Enter a number: ";
    cin >> number;
    // Use conditional (ternary) operator to calculate
    absolute value
    number = (number < 0)? -number : number;
    cout << "\n The absolute value is: " << number ;

    return 0;
}
```

```
int main()
{
    int number;
    cout << "Enter a number: ";
    cin >> number;
    // Use conditional (ternary) operator to check if the
    number is even or odd
    number % 2 == 0 ? cout << "even" : cout << "odd";
    // cout << (number % 2 == 0 ? "even" : "odd");
    return 0;
}
```

مثال



```
if ((x > 5) && (x < 10))  
    cout << x << endl ;
```

اگر x از ۵ بزرگتر و از ۱۰ کوچکتر باشد، آنگاه مقدار x نمایش داده شود.

```
if (x)  
    cout << x << endl;
```

اگر مقدار x مخالف صفر باشد، آنگاه مقدار x نمایش داده شود.

نکته: در $C++$ هر مقدار غیر صفر به منزله درست (**true**) و مقدار صفر به منزله نادرست (**false**) ارزیابی می شود.

دستورالعملهای شرطی تو در تو

Nested if ... else ... statements



از دستورالعمل شرطی (if-else) می توان به صورت تو در تو استفاده نمود؛ یعنی در صورت نیاز درون دستور if، از دستور if دیگری استفاده نمود.

فرض کنید که می خواهیم نمرات دانشجویان که اعدادی بین ۰ تا ۱۰۰ بوده است را به نمرات حرفی تبدیل کنیم.
به این شکل که نمرات بالای ۹۰ معادل A،
نمرات بین ۸۰ تا ۹۰ معادل B،
نمرات بین ۷۰ تا ۸۰ معادل C،
نمرات بین ۶۰ تا ۷۰ معادل D و
سایر نمرات معادل F باشد.

```
if (studentGrade >= 90) {  
    cout << "A";  
}  
else {  
    if (studentGrade >= 80) {  
        cout << "B";  
    }  
    else {  
        if (studentGrade >= 70) {  
            cout << "C";  
        }  
        else {  
            if (studentGrade >= 60) {  
                cout << "D";  
            }  
            else {  
                cout << "F";  
            }  
        }  
    }  
}
```

دستورالعملهای شرطی تو در تو

Nested if ... else ... statements



- برنامه‌نویسان معمولاً ترجیح می‌دهند، برای خوانایی و ظاهر بهتر برنامه دستورالعملهای شرطی تو در تو را به شکل زیر بنویسند. این شیوه نوشتن تفاوتی با شکل قبلی ندارد؛ فقط در فضاهای خالی که توسط کامپایلر نادیده گرفته می‌شود متفاوت است.
- دقت کنید که هنگام نوشتن دستورالعملهای شرطی همه حالت‌های ممکن را مورد توجه قرار دهید.

```
if (studentGrade >= 90) {  
    cout << "A";  
}  
else if (studentGrade >= 80) {  
    cout << "B";  
}  
else if (studentGrade >= 70) {  
    cout << "C";  
}  
else if (studentGrade >= 60) {  
    cout << "D";  
}  
else {  
    cout << "F";  
}
```

فرض کنید که می‌خواهیم نمرات دانشجویان که اعدادی بین ۰ تا ۱۰۰ بوده است را به نمرات حرفی تبدیل کنیم. به این شکل که نمرات بالای ۹۰ معادل A، نمرات بین ۸۰ تا ۹۰ معادل B، نمرات بین ۷۰ تا ۸۰ معادل C، نمرات بین ۶۰ تا ۷۰ معادل D و سایر نمرات معادل F باشد.

دستورالعملهای شرطی چند انتخابی

Multiple-selection statements



در زبان C++ به غیر از دستورالعمل شرطی (if-else)، دستورالعمل switch که عملاً یک دستورالعمل چند انتخابی می باشد، ارائه شده است. در این دستورالعمل وابسته به مقدار عبارت یکی از چندین دستورالعمل (یا مجموعه ای از دستورالعملها) انتخاب و اجرا می شود.

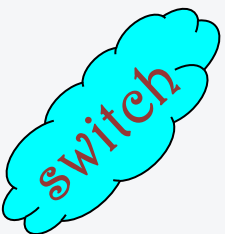
```
switch(expression)
{
    case constant1:
        statement;
        ...
        statement ;
        break;

    case constant2:
        statement;
        ...
        statement;
        break;

    ...

    default:
        statement;
        ...
        statement;

}
```



مثال



مثال ۴: برنامه‌ای بنویسید که عددی را از ورودی دریافت کند، اگر عدد دریافتی ۰، ۱ و ۲ باشد به ترتیب zero، one و two را نمایش دهد و در غیر این صورت other را نمایش دهد.

```
#include <iostream.h>
using namespace std;

int main()
{
    unsigned int number;
    cout << "Enter a unsigned number: ";
    cin >> number;
    // use switch statement to check value of the number
    switch (number)
    {
        case 0:                                // if the number is 0
            cout << "zero" << endl;
            break;                             // exit switch
        case 1:                                // if the number is 1
            cout << "one" << endl;
            break;                             // exit switch
        case 2:                                // if the number is 2
            cout << "two" << endl;
            break;
        default:                               // if the number is not 0, 1, or 2
            cout << "other" << endl;
    }    /* end of switch statement */
    return 0;
}
```

مثال



مثال ۵: برنامه‌ای بنویسید که عدد صحیح مثبتی را از ورودی دریافت کند، و با استفاده از دستور switch اگر عدد دریافتی کمتر از ۳ باشد پیام Less than Three، اگر عدد دریافتی ۳ باشد پیام Equal to Three و اگر بزرگتر از ۳ باشد پیام Greater than Three را چاپ کند.

```
#include <iostream.h>
using namespace std;

int main()
{
    unsigned int n;
    cout << "Enter a unsigned number: ";
    cin >> n;
    // use switch statement to check value of the number
    switch(n)
    {
        case 0: // if the number is 0
        case 1: // if the number is 1
        case 2: // if the number is 2
            cout << "Less Than Three" << endl;
            break; // exit switch
        case 3: // if the number is 3
            cout << "Equal To Three" << endl;
            break; // exit switch
        default: // if the number is not 0, 1, 2, or 3
            cout << "Greater Than Tree" << endl;
    } // * end of switch statement */
    return 0;
}
```

تمرین



1. برنامه‌ای بنویسید که ضلع مربع را دریافت کند و محیط و مساحت مربع را محاسبه کند.
2. برنامه‌ای بنویسید که طول و عرض مستطیل را دریافت کند و محیط و مساحت مستطیل را محاسبه کند.
3. برنامه‌ای بنویسید که عددی از ورودی دریافت کند اگر عدد سه رقمی است «Yes» و اگر عدد سه رقمی نیست «No» را چاپ کند.
4. برنامه‌ای بنویسید که سه عدد حقیقی را از ورودی دریافت کند و کوچکترین عدد را چاپ کند.
5. برنامه‌ای بنویسید که سه عدد حقیقی را از ورودی دریافت کند و اعداد را از کوچک به بزرگ چاپ کند.
6. برنامه‌ای بنویسید که طول سه پاره‌خط را از ورودی دریافت کند و مشخص کند که آیا این پاره‌خط‌ها تشکیل یک مثلث می‌دهد یا خیر.
7. برنامه‌ای بنویسید که عددی از ورودی دریافت کند و ماه متناظر با عدد را چاپ کند. (یعنی برای عدد ۱ فروردین، ۲ اردیبهشت و ... را چاپ کند. برای اعداد کمتر از یک و بزرگتر از ۱۲ پیغام خطا را چاپ کند).

حلقه تکرار for

for Iteration Loop

از دستور العمل for برای تکرار یک یا چند دستورالعمل استفاده می شود. شکل کلی دستور for بصورت زیر می باشد:

```
for (initialization; condition; update)  
    statement;
```

```
for (initialization; condition; update)  
{  
    statement;  
    statement;  
    :  
    statement;  
}
```



for

حلقه تکرار for

for Iteration Loop



```
for ( initialization; condition; update )  
{  
    statements;  
}
```

```
int i;  
for (i=1; i<=3; i++)  
{  
    cout << "hello \n";  
}
```

مثال

مثال

مثال ۶: برنامه‌ای بنویسید که کلمه "hello" را سه بار چاپ کند.

```
#include <iostream.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int i;
```

```
    /* Use a for loop to print "hello" three times*/
```

```
    for(i = 1; i <= 3; i++)
```

```
    {
```

```
        cout << "hello\n";
```

```
    }
```

```
    return 0;
```

```
}
```

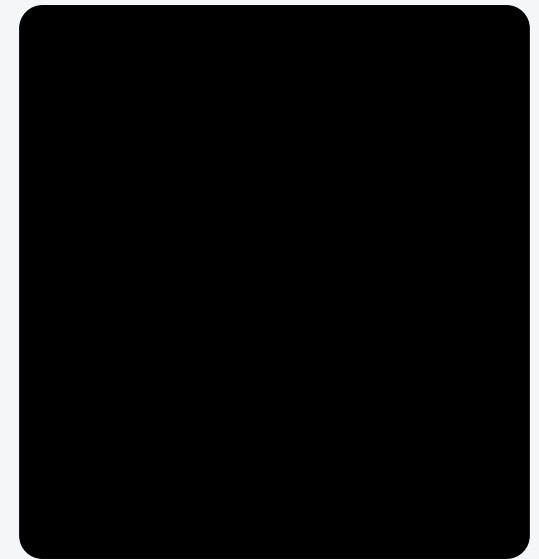
حلقه تکرار for

for Iteration Loop

```
int i;
```

```
for (i=1; i<=3; i++)  
{  
    cout << "hello \n";  
}
```

i =



حلقه تکرار for

for Iteration Loop



```
int i;  
for (i=1; i<=3; i++)  
{  
    cout << "hello \n";  
}
```

A red wavy line encircles the body of the for loop. Red arrows indicate the flow of execution: one arrow points from the variable 'i' in the declaration to its first use in the loop condition, and another arrow points from the 'i++' part of the loop condition back to the 'i' in the declaration, showing the iterative nature of the loop.

i =  2

hello

حلقه تکرار for

for Iteration Loop



```
int i;
```

```
for (i=1; i<=3; i++)  
{  
    cout << "hello \n";  
}
```

A red wavy line with arrows indicating the loop iteration process. It starts from the 'i++' part of the for loop, goes down to the closing brace '}', then loops back up to the 'i=1' part of the for loop, and continues to loop back to the 'i++' part, illustrating the repeating nature of the loop.

i =   3

```
hello  
hello
```

حلقه تکرار for

for Iteration Loop



```
int i;
```

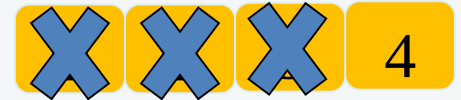
```
for (i=1; i<=3; i++)
```

```
{
```

```
    cout << "hello \n";
```

```
}
```

i =



hello

hello

hello

حلقه تکرار for

for Iteration Loop



```
int i;
```

```
for (i=1; i<=3; i++)
```

```
{
```

```
    cout << "hello \n";
```

```
}
```

i =  4

hello

hello

hello

حلقه تکرار for

for Iteration Loop



- اعلان شمارنده حلقه می تواند در داخل دستور for باشد.

```
for(int i = 1;i <= 3;i++)  
{  
    cout << "hello\n";  
}
```

- لزومی ندارد که شمارنده حلقه حتماً از ۱ شروع شود.

```
int i;  
  
for(i = 5;i <= 7;i++)  
{  
    cout << "hello\n";  
}
```

حلقه تکرار for

for Iteration Loop



- مقدار دهی اولیه شمارنده حلقه می تواند خارج از دستور **for** باشد. در این صورت می توان عبارت اول در حلقه **for** را خالی گذاشت.

```
int i = 1;

for( ;i <= 3;i++)
{
    cout << "hello\n";
}
```

- تغییرات شمارنده می تواند در بدنه دستور **for** تعریف شود. در این صورت می توان عبارت سوم در حلقه **for** را خالی گذاشت.

```
int i

for(i = 1 ;i <= 3; )
{
    cout << "hello\n";
    i++;
}
```

حلقه تکرار for

for Iteration Loop



- لزومی ندارد که تغییرات شمارنده بصورت افزایشی باشد بلکه می تواند به صورت کاهشی نیز باشد.

```
int i;  
  
for(i = 3 ;i >= 1;i--)  
{  
    cout << "hello\n";  
}
```

- تغییرات شمارنده می تواند افزایش یا کاهش بیش از ۱ واحد داشته باشد.

```
int i  
  
for(i = 1 ;i <= 3; i = i+2)  
{  
    cout << "hello\n";  
}
```

```
int i  
  
for(i = 1 ;i <= 3; i +=2)  
{  
    cout << "hello\n";  
}
```

حلقه تکرار for

for Iteration Loop



- در دستور for اگر قسمت شرط خالی باشد، هیچ شرطی برای توقف نداریم. در صورتی این کار منطقی است که درون بدنه حلقه شرطی برای خروج داشته باشیم.

```
int i;  
  
for(i = 1 ; i++)  
{  
    cout << "hello\n";  
}
```

- شمارنده حلقه می‌تواند متغیر کاراکتری یا اعشاری باشد و لزومی ندارد حتما شمارنده عدد صحیح باشد.

```
char i  
  
for(i = 'a' ; i <= 'c' ; i++)  
{  
    cout << "hello\n";  
}
```

تمرین



1. برنامه‌ای بنویسید که n را از ورودی دریافت کند و مجموع 1 تا n را محاسبه کند.
 2. برنامه‌ای بنویسید که 100 عدد را از ورودی دریافت کند و میانگین آنها را محاسبه کند.
 3. برنامه‌ای بنویسید دو عدد از ورودی دریافت کند و اعداد زوج (فرد، مضارب 5) بین این دو عدد را چاپ کند.
 4. برنامه‌ای بنویسید که 100 عدد را از ورودی دریافت کند و کوچکترین و بزرگترین آنها را چاپ کند.
 5. برنامه‌ای بنویسید که n را از ورودی دریافت کند و مشخص کند که عدد اول است یا نه.
 6. برنامه‌ای بنویسید که n را از ورودی دریافت کند و n امین جمله دنباله فیبوناچی را محاسبه کند.
- 1, 1, 2, 3, 5, 8, ...
8. برنامه‌ای بنویسید که x و y را از ورودی دریافت کند و x^y را محاسبه کند.
 9. برنامه‌ای بنویسید که عددی را از ورودی دریافت کند و تعداد ارقام آن را چاپ کند.
 01. برنامه‌ای بنویسید که عددی را از ورودی دریافت کند و مقلوب آن را چاپ کند.

3761 $\rightarrow$ 1673

مثال



مثال ۶: برنامه‌ای بنویسید که عدد صحیح و مثبت n را از ورودی گرفته فاکتوریل آن را محاسبه و نمایش دهد.

```
#include <iostream.h>

using namespace std;

int main()
{
    unsigned int n, i;
    long long int fact = 1;

    cout << "Enter a positive integer number:";
    cin >> n;
    for( i = 1; i <= n; ++i)
        fact *= i;
    cout << n << "!=" << fact << endl;
    return 0;
}
```

دستورالعمل break

break statement

- اگر در بدنه حلقه for بنا به شرایطی بخواهیم ادامه تکرار حلقه را متوقف کنیم، می توان از دستور break استفاده کرد، این دستور ادامه ی اجرای حلقه را متوقف می کند و روند اجرا را به اولین دستورالعمل بعد از انتهای حلقه منتقل می نماید.

```
int i;  
  
for(i = 1 ; i < 10 ;i++)  
{  
    if (i == 4) break;  
    cout << i << endl;  
}
```

```
1  
2  
3
```

break

دستورالعمل continue

continue statement



اگر این دستورالعمل درون حلقه استفاده شود، باعث می‌شود که اجرا به ابتدای حلقه منتقل شود. شمارنده بر اساس دستورالعمل حلقه به‌هنگام می‌شود؛ سپس شرط حلقه بررسی شده و در صورت درست بودن شرط، حلقه ادامه می‌یابد و در غیر این صورت تکرار حلقه متوقف می‌شود.

```
int i;  
  
for(i = 1 ; i < 10 ;i++)  
{  
    if (i == 4) continue;  
    cout << i << endl;  
}
```

```
1  
2  
3  
5  
6  
7  
8  
9
```

continue

مثال



مثال ۷: برنامه‌ای بنویسید که حداکثر ۱۰۰ عدد را از کاربر دریافت کند و میانگین آنها را محاسبه کند، در صورتی کاربر در حین ورود اعداد صفر را وارد کند، این کار به معنی اتمام اعداد تلقی شود.

```
#include <iostream.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int n, i, sum = 0;
```

```
    float average;
```

```
    for( i = 0; i < 100; ++i)
```

```
    {
```

```
        cout << "Enter a number:";
```

```
        cin >> n;
```

```
        if(n==0)
```

```
            break;
```

```
        sum += n; // sum = sum + n
```

```
    }
```

```
    average = (float) sum / i;
```

```
    cout << "average = " << average << endl;
```

```
    return 0 ;
```

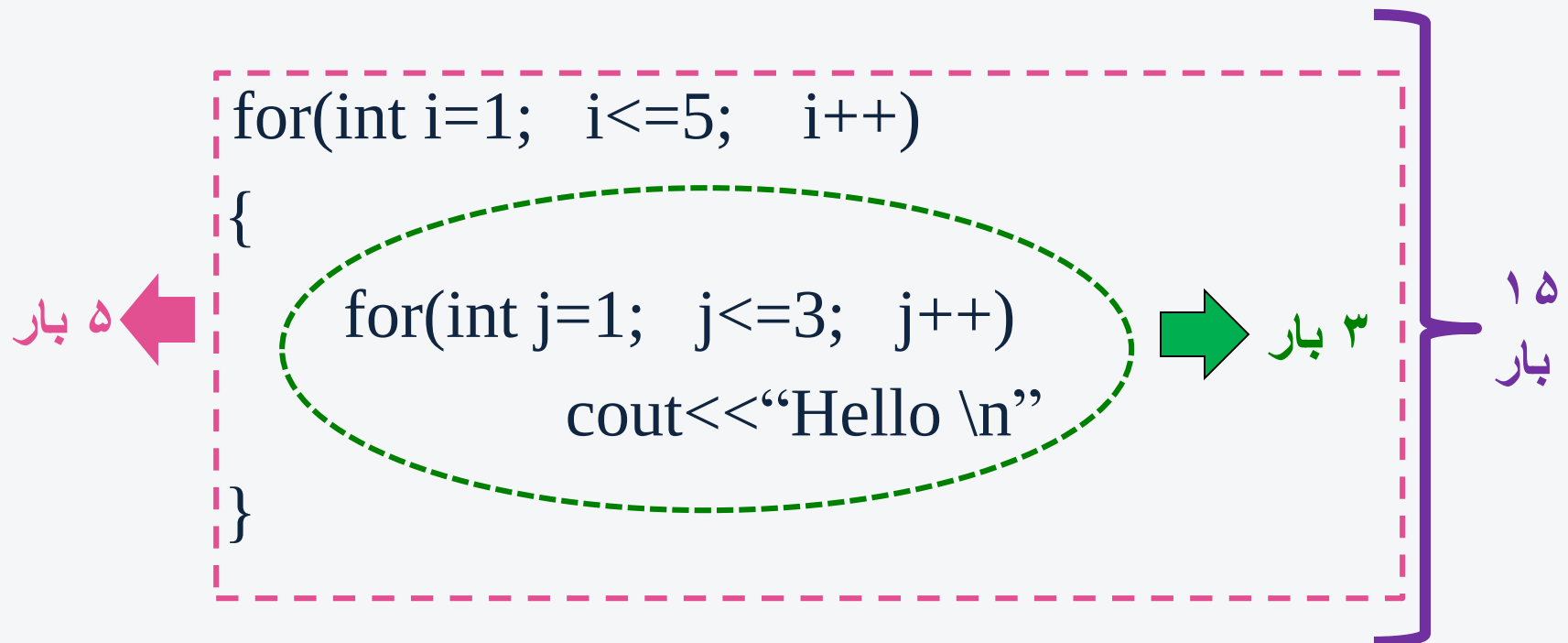
```
}
```

حلقه های for تو در تو

Nested for Loop



- می توان داخل بدنه دستور **for** هر دستور دلخواهی را بسته به نیاز به کار برد. پس در صورت نیاز می توان از یک دستور **for** دیگر در بدنه دستور **for** استفاده کرد.
- قطعه کد زیر عبارت **Hello** را ۱۵ بار چاپ می کند.



حلقه تکرار while

while Iteration Loop

این دستورالعمل مانند دستورالعمل **for** برای تکرار یک یا چند دستورالعمل استفاده می‌گردد. دستور **while** برخلاف دستور **for** شمارنده صریح ندارد و بیشتر هنگامی که تعداد تکرار حلقه نامشخص است، استفاده می‌شود. شکل کلی این دستورالعمل به صورت زیر می‌باشد.

```
while (condition)  
statement;
```

```
while (condition)  
{  
    statement;  
    statement;  
    :  
    statement;  
}
```



while

مثال



مثال ۸: برنامه ای بنویسید که عددی را از ورودی دریافت کند و تعداد ارقام آن را چاپ کند.

```
#include <iostream.h>

using namespace std;

int main()
{

    long int x;
    short int k = 0;
    cout << "Enter a number:";
    cin >> x;

    while(x > 0)
    {
        x = x / 10;
        k++;
    }
    cout << "\n" << k;

    return 0;
}
```

حلقه تکرار do-while

do-while Iteration Loop

این دستورالعمل مانند دستورالعمل **for** و **while** برای تکرار یک یا چند دستورالعمل استفاده می‌گردد. در دستور **do-while** برخلاف دستور **while** شرط حلقه در انتهای حلقه قرار دارد، پس دستورات درون حلقه حداقل یک بار اجرا می‌شود. شکل کلی این دستورالعمل به صورت زیر می‌باشد.

```
do  
    statement;  
while (condition);
```

```
do  
{  
    statement;  
    statement;  
    :  
    statement;  
} while (condition);
```



**do
while**

مثال



مثال ۹: برنامه ای بنویسید که تا زمانی که صفر وارد نشده است، اعداد را از ورودی دریافت کند و مجموع آنها را محاسبه کند.

```
#include <iostream.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int x;
```

```
    int sum = 0;
```

```
    do
```

```
    {
```

```
        cin >> x;
```

```
        sum = sum + x;
```

```
    } while(x != 0);
```

```
    cout << '\n' << sum;
```

```
    return 0;
```

```
}
```

حلقه‌های تکرار

do-while Iteration Loop



- دستور **while** را می‌توان به جای دستور **for** به کار برد.

```
int i = 1;
while(i <= 3)
{
    cout << "hello\n";
    i++;
}
```

- در دستور **while** و **do-while** هم مانند دستور **for** می‌توان از دستور **break** استفاده کرد. در صورت استفاده از **break** اجرا به دستور بعد از انتهای حلقه منتقل می‌شود.
- در دستور **while** و **do-while** هم مانند دستور **for** می‌توان از دستور **continue** استفاده کرد. در صورت استفاده از **continue** اجرا دستور به ابتدای حلقه منتقل می‌شود و شرط مجددا بررسی می‌شود.
- حلقه‌های مختلف را می‌توان در صورت نیاز به شکل تودرتو به کار برد.

تمرین



1. برنامه‌ای بنویسید عددی را از ورودی دریافت کند و تمامی اعداد اول کوچکتر از آن را چاپ کند.
2. برنامه‌ای بنویسید که جدول ضرب اعداد یک رقمی را چاپ کند.
3. برنامه‌ای بنویسید که تمامی اعداد سه رقمی که می‌توان با سه رقم ۵، ۷ و ۹ ایجاد کرد را چاپ کند.
4. برنامه‌ای بنویسید که عددی را دریافت کند و آن را به مبنای ۲ تبدیل کند.
5. برنامه‌ای بنویسید که که عددی در مبنای ۲ را دریافت کند و آن را به مبنای ۱۰ تبدیل کند.
6. برنامه‌ای بنویسید با گرفتن n شکلی مشابه زیر ایجاد کند که سطر آخر n ستاره داشته باشد.

```
*
**
***
****
*****
*****
*****
```

7. برنامه‌ای بنویسید با گرفتن n (یک عدد فرد) شکلی مشابه زیر ایجاد کند که سطر آخر n ستاره داشته باشد.

```
  *
 ***
*****
*****
*****
*****
```

با سپاس از توجه شما

